Programming Basics · Study Notes

Demo notes generated for studyfold.

1. Variables and types

A variable is a name that points to a value. Remember that **every value has a type**, and the type decides which operations are allowed. When in doubt, check it with `type(x)` in the console.

> **Key idea:** assignment creates a name; it never copies the object.

- Names should describe what a value means, not how it is stored.
- Small functions are easier to test than long scripts.
- Read the error message from the bottom line upwards.

```
price = 19.99
count = 3
total = price * count
print(f"Total: {total:.2f}")
```

In the exam, explain *why* your solution works, not only what it prints. A short comment above a tricky line is often enough.

Concept	Example	Common mistake
List	`[1, 2, 3]`	Changing it while looping over it
Dictionary	`{"a": 1}`	Assuming the key always exists

2. Conditions

Programs make decisions with `if`, `elif` and `else`. Remember that **every value has a type**, and the type decides which operations are allowed. When in doubt, check it with `type(x)` in the console.

> **Key idea:** only the first matching branch runs.

- Names should describe what a value means, not how it is stored.
- Small functions are easier to test than long scripts.
- Read the error message from the bottom line upwards.

```
def grade(score):
    if score >= 9:
        return "A"
    elif score >= 5:
        return "Pass"
    return "Fail"
```

In the exam, explain *why* your solution works, not only what it prints. A short comment above a tricky line is often enough.

Concept	Example	Common mistake
List	`[1, 2, 3]`	Changing it while looping over it
Dictionary	`{"a": 1}`	Assuming the key always exists

3. Loops

A loop repeats a block of code for each item of a sequence. Remember that **every value has a type**, and the type decides which operations are allowed. When in doubt, check it with `type(x)` in the console.

Key idea: use `for` when you know the items, `while` when you wait for a condition.

- Names should describe what a value means, not how it is stored.
- Small functions are easier to test than long scripts.
- Read the error message from the bottom line upwards.

```
words = ["data", "cloud", "python"]
for i, w in enumerate(words, start=1):
    print(i, w.upper())
```

In the exam, explain *why* your solution works, not only what it prints. A short comment above a tricky line is often enough.

Concept	Example	Common mistake
List	`[1, 2, 3]`	Changing it while looping over it
Dictionary	`{"a": 1}`	Assuming the key always exists

4. Functions

A function packs a task behind a name, with inputs and an output. Remember that **every value has a type**, and the type decides which operations are allowed. When in doubt, check it with `type(x)` in the console.

Key idea: a function without `return` gives back `None`.

- Names should describe what a value means, not how it is stored.
- Small functions are easier to test than long scripts.
- Read the error message from the bottom line upwards.

```
def average(values):
    if not values:
        return 0
    return sum(values) / len(values)
```

In the exam, explain *why* your solution works, not only what it prints. A short comment above a tricky line is often enough.

Concept	Example	Common mistake
List	`[1, 2, 3]`	Changing it while looping over it
Dictionary	`{"a": 1}`	Assuming the key always exists

5. Dictionaries

A dictionary maps keys to values, like a small database table. Remember that **every value has a type**, and the type decides which operations are allowed. When in doubt, check it with `type(x)` in the console.

> **Key idea:** use `.get(key, default)` when a key may be missing.

- Names should describe what a value means, not how it is stored.
- Small functions are easier to test than long scripts.
- Read the error message from the bottom line upwards.

```
stock = {"apples": 4, "pears": 0}
for fruit, n in stock.items():
    print(fruit, "OK" if n else "empty")
```

In the exam, explain *why* your solution works, not only what it prints. A short comment above a tricky line is often enough.

Concept	Example	Common mistake
List	`[1, 2, 3]`	Changing it while looping over it
Dictionary	`{"a": 1}`	Assuming the key always exists